Building a Real-Time IP Camera Viewer with Python & OpenCV

Individual Assignment Report & Testing Log

1. Project Overview & Implementation Notes

For this assignment, I built a lightweight desktop application that connects to an IP camera stream on my local Wi-Fi network and displays the live feed without requiring proprietary camera software. I used an Android phone running the IP Webcam app as the hardware camera source, streaming MJPEG video over HTTP.

The core application was built in Python using OpenCV for frame ingestion and image processing. Along with rendering the live video, the program resizes incoming frames while maintaining aspect ratio, overlays real-time telemetry (camera name, display resolution, timestamp, and a smoothed FPS counter), captures timestamped screenshots on pressing **S**, records video on pressing **R**, and safely manages network drops and process termination without leaving behind corrupted video files.

```
ip-camera-streamer/ ├── config.py # Reads .env, sets defaults, and verifies URL structure
├── stream_processor.py # Resizes frames, smooths FPS with EMA, and renders HUD overlays ├──
main.py # Main frame loop, keyboard input handler, and signal traps ├── screenshots/ # Saved
JPEG stills with microsecond timestamps └── recordings/ # Saved MP4 recordings (encoded with
FourCC 'mp4v')
```

2. Test Execution & Results Matrix (TC01 – TC10)

I tested each requirement against the running application and documented the behavior. All 10 test cases passed without program crashes or unhandled exceptions.

ID	Test Scenario	What Happened During Testing	Result	Evidence
TC01	Live stream launch	Stream opened in about 1.5s; video rendered smoothly in a desktop window.	PASSED	TC01_live.png
TC02	Malformed stream URL	Passed 'htp://camera' in .env; printed a clean error message and exited with code 1.	PASSED	TC02_err.png
TC03	Camera offline / wrong IP	Preflight HTTP check timed out after 5 attempts (10s total) and shut down cleanly.	PASSED	TC03_fail.png
TC04	HUD Telemetry overlays	Camera name, resolution, clock, and smoothed FPS stayed clear and readable.	PASSED	TC04_ovl.png
TC05	Proportional resizing	Scaled 1080p down to 960×540 without stretching or squeezing circular shapes.	PASSED	TC05_geom.png
TC06	Screenshot capture (S)	Saved two unique files to screenshots/ with a 2-second green banner confirming each.	PASSED	TC06_cap.png
TC07	Video recording (R)	Red REC dot appeared; recorded for 10 seconds; the MP4 file played cleanly in VLC.	PASSED	TC07_rec.png
TC08	Network drop mid-stream	Toggled phone Wi-Fi off for 6s; app logged dropped frames and resumed when Wi-Fi returned.	PASSED	TC08_drop.png

ID	Test Scenario	What Happened During Testing	Result	Evidence
TC09	Esc key during recording	Pressed Esc while recording; writer released properly, and the saved video was playable.	PASSED	TC09_esc.png
TC10	Ctrl+C during recording	Sent SIGINT in terminal; caught the interrupt signal, saved the file, and exited with no traceback.	PASSED	TC10_trap.png

3. Technical Questions: Part A — Python and Networking

1. What is an IP address, and why must your computer and the camera be on the same local network for this project to work?

An IP address is the unique numerical address assigned to a device on a network so other devices can send data packets to it. In my setup, the computer and phone must share the same local Wi-Fi subnet (like `192.168.1.0/24`) because they use private IP addresses. Private IPs cannot be routed across the public internet. By staying on the same local network, my computer can talk directly to the phone using local ARP address lookups without needing complex router port-forwarding or dynamic DNS configuration.

2. Explain, step by step, what happens when your program opens a stream URL such as http://192.168.1.23:8080/video. What part does HTTP play?

First, the computer checks if `192.168.1.23` is on its local subnet and looks up its hardware MAC address using an ARP broadcast. Next, it establishes a reliable TCP connection to port 8080 with a standard 3-way handshake (SYN, SYN-ACK, ACK). Then my program sends an HTTP `GET /video` request. HTTP acts as the communication wrapper here: the phone replies with an `HTTP 200 OK` status and a `multipart/x-mixed-replace` header. Instead of closing the connection after one page, HTTP keeps the socket open and continuously delivers consecutive JPEG image frames separated by MIME boundary strings.

3. What is a port number? Why does the camera's URL include :8080, and what would happen if you left it out?

A port number is a numerical channel (0 to 65535) that directs network traffic to a specific software program running on a device. The URL specifies `:8080` because the IP Webcam app runs as an ordinary mobile application; on Android/Linux, only root system services are allowed to use standard privileged ports below 1024 (like port 80). If I left out `:8080`, Python would assume the standard HTTP port 80 by default and immediately fail with a *Connection Refused (Error 10061)* because nothing is listening on port 80.

4. What is the difference between an MJPEG stream over HTTP and an RTSP stream? Which did your camera provide, and how did you know?

An MJPEG stream is simply a continuous sequence of standalone JPEG pictures sent over an HTTP connection. It takes more network bandwidth because every frame is a full image with no compression between frames, but it has very little lag and is straightforward to decode. RTSP is a dedicated streaming control protocol that negotiates sessions while sending video data via RTP using codecs like H.264, which compress video by only sending changes between frames. My phone camera provided an **MJPEG over HTTP** stream. I verified this by checking the stream URL scheme (`http://`) and probing the headers with Requests, which returned `Content-Type: multipart/x-mixed-replace`.

5. Why must the camera's address and any username or password be kept out of your source code? Show how your project keeps them out.

Hardcoding local IPs, usernames, or passwords in Python files is risky because committing them to GitHub exposes my internal network layout and credentials to anyone viewing the repository. In my project, I created an untracked `.env` file:

```
CAMERA_STREAM_URL=http://192.168.1.128:8080/video CAMERA_NAME=Office_Cam_01
```

My code loads these values with `python-dotenv` inside `config.py`. I added `.env` directly to `.gitignore`, and provided a safe template named `.env.example` with dummy placeholders for version control.

4. Technical Questions: Part B — Python Libraries

1. What does cv2.VideoCapture do? What do its isOpened() and read() methods return, and how does your code use each return value?

`cv2.VideoCapture` is the OpenCV class responsible for connecting to a camera device, local video file, or network video stream. Its `isOpened()` method returns a boolean indicating whether the stream was successfully initialized; my code checks this before starting the main loop to decide if it needs to attempt a reconnection. The `read()` method grabs and decodes the next frame, returning a tuple: `(ret, frame)`. `ret` is a boolean that tells if the read worked, and `frame` is the image array itself. If `ret` is `False`, my program increments a dropped frame counter to detect when the camera has disconnected.

2. A frame from OpenCV is a NumPy array. What does its shape tell you, and what is the shape of one 1280 × 720 colour frame?

The array's `shape` tuple describes the dimensions of the image in the order of `(Height, Width, Color Channels)`. The height is the number of pixel rows, width is the number of pixel columns, and channels represent color depth. A standard 1280 × 720 color frame produces a shape of **(720, 1280, 3)**, meaning 720 rows, 1280 columns, and 3 color channels.

3. OpenCV stores colours in BGR order rather than RGB. Where in your project does that matter, and what would go wrong if you forgot it?

This matters whenever I draw text or shapes using functions like `cv2.putText()`, `cv2.circle()`, or `cv2.rectangle()`. When drawing a red indicator dot, I had to supply `(0, 0, 255)` rather than the usual RGB `(255, 0, 0)`. If I forgot this channel order or saved frames with a library that expects standard RGB (like Pillow or Matplotlib), the colors would look completely wrong: red recording dots would turn blue, and people's faces would take on an unnatural blue-green tint.

4. What is cv2.waitKey() for, and why does the window stop updating without it? How did you use it to detect Esc, S and R?

`cv2.waitKey()` serves two essential roles: it pauses execution for a brief moment to listen for keyboard keypresses, and it drives the operating system's window manager to process GUI redraw and paint events. Without it, the operating system thinks the window has frozen and stops updating the video feed. I read keypresses using `key = cv2.waitKey(1) & 0xFF`. I checked for Esc with `key == 27`, S with `key in (ord('s'), ord('S'))`, and R with `key in (ord('r'), ord('R'))`.

5. What did you use the Requests library for? Why is it useful to check that the camera answers before handing the URL to OpenCV?

I used `requests.get(url, stream=True, timeout=5)` to perform a fast preflight health check before initializing OpenCV. This is helpful because if the phone is turned off, on a different Wi-Fi network, or typing the wrong IP address, passing that URL directly to `cv2.VideoCapture` can cause the underlying FFmpeg driver to hang the whole program for 30 to 60 seconds without feedback. The Requests check fails fast in 5 seconds and gives immediate console feedback.

5. Technical Questions: Part C — Video Processing

1. How does your program estimate FPS? Why does the number change from frame to frame, and how could you make it steadier?

My code estimates FPS by timing the interval between frames using Python's high-precision clock (`time.perf_counter()`) and taking `1.0 / (current_time - previous_time)`. The instant FPS value jumps around constantly because of small Wi-Fi packet delays, variable phone frame processing, and operating system background tasks. To keep the on-screen display steady and readable, I applied an Exponential Moving Average (EMA) with a 0.9 smoothing factor:
`Smoothed_FPS = (0.9 × Smoothed_FPS) + (0.1 × Instant_FPS)`
This smoothed out sudden spikes while still reflecting real performance drops.

2. How did you resize frames while keeping their aspect ratio? Show the calculation for resizing a 1920 × 1080 frame to a width of 960.

To resize proportionally, I calculated a scaling factor by dividing the desired display width by the frame's original width, then multiplied the original height by that same factor:
`Scale Factor = 960 / 1920 = 0.5`
`Target Height = int(1080 × 0.5) = 540`
The frame was resized using `cv2.resize(frame, (960, 540))`. Because both dimensions were cut in half, the original 16:9 widescreen proportions remained intact without any visual distortion.

3. What does cv2.VideoWriter need — codec (FourCC), frame rate and frame size — and what happens if the frames you write are a different size from the one you declared?

`cv2.VideoWriter` needs an output filename, a FourCC compression codec code (such as `* 'mp4v'`), the target playback frame rate (like 30.0), and the frame dimensions given as **(Width, Height)**. Notice that OpenCV expects `(Width, Height)` here, which is the exact opposite of NumPy's `(Height, Width)` array indexing. If the frame array passed to `write()` does not match the declared dimensions down to the pixel, OpenCV silently drops the frame without throwing an error, resulting in an unplayable 0 KB file.

4. Why does drawing overlays and writing a recording on every frame cost performance? Name two ways to reduce that cost.

Overlaying text and banners forces the CPU to rasterize font characters and blend pixel colors across the frame array on every iteration. Video encoding is even heavier because it converts colors and runs mathematical compression algorithms before writing data to disk. Two practical ways to reduce this cost are:
1. Separate Threads for I/O: Use a worker thread for video capture and disk writing, sharing frames via a `queue.Queue` so network lag or disk writing never slows down the display loop.
2. Limit Overlay Recalculations: Instead of recomputing text and backgrounds on all 30 frames every second, update text strings only once every few frames while reusing cached overlay coordinates.

5. How do you name screenshot and recording files so that a new one never overwrites an old one?

I generated filenames using date and time formatting down to the microsecond level for screenshots, and to the second for recordings:

```
screenshot_name = f"screenshots/screenshot_{datetime.now().strftime('%Y%m%d_%H%M%S_%f')}.jpg"
recording_name = f"recordings/recording_{datetime.now().strftime('%Y%m%d_%H%M%S')}.mp4"
```

Because the clock continually moves forward, every captured file gets a distinct filename, so rapid keypresses never overwrite existing files.

6. Technical Questions: Part D — Critical Thinking

1. What does your program do if the Wi-Fi drops for ten seconds in the middle of streaming? Describe your reconnect logic and why you chose its limits.

When Wi-Fi disconnects, `cap.read()` starts failing and returns `ret=False`. My code counts consecutive failed reads; once it hits 30 dropped frames (roughly 1 second of lost feed), it flags the drop. It releases the old capture handle, pauses 1 second to let Windows clear the broken socket, and tries reconnecting up to 5 times with a 2-second delay between attempts. This gives the camera an 11-to-12-second window to reconnect, which is enough to survive brief Wi-Fi hiccups without leaving the program hanging indefinitely if the camera is powered down permanently.

2. Why must the camera and the VideoWriter be released on every way out of the program? What happens to a recording that is never released?

Calling `release()` frees network sockets and camera hardware handles back to the operating system. For `VideoWriter`, calling `release()` is essential because MP4 container formats keep index metadata (the `moov` atom) in memory while recording. That metadata is only written to the file header when the writer is closed cleanly. If the script crashes or closes without releasing the writer, that header is never saved, making the video file corrupt and unplayable in media players like VLC.

3. What privacy and ethical responsibilities come with building a tool that streams and records video? How does your design respect them?

Camera software can easily be misused for unauthorized monitoring or recording people without their knowledge and consent. My implementation respects privacy by keeping all video data strictly on the local Wi-Fi network without relaying footage to third-party cloud servers. Sensitive credentials and camera URLs are kept in a local `.env` file that is excluded from Git, a bright red ● REC indicator appears on-screen while recording so anyone viewing the monitor knows capture is active, and both the `screenshots/` and `recordings/` folders are gitignored to avoid uploading surveillance media to a public repository.

4. If this viewer had to show ten cameras at once, what would fail first, and how would you redesign it?

Running ten cameras inside a single-threaded loop would fail immediately. Each camera's `cap.read()` call blocks waiting for network packets; reading ten cameras sequentially would introduce massive latency and cause the overall frame rate to drop to an unusable 1 to 2 frames per second. To fix this, I would redesign the system using a multi-threaded architecture: assign each camera its own worker thread that writes only the newest frame to a single-slot buffer (`collections.deque(maxlen=1)`). The main UI thread would then grab the latest frame from each buffer and tile them into a single 2×5 grid using NumPy array slicing.

5. What was the hardest bug you met in this project? How did you find its cause, and what would you do differently next time?

The trickiest bug I encountered was when `cv2.VideoWriter` created 0 KB video files that could not be opened, even though the video looked fine on screen. I realized that OpenCV's VideoWriter constructor expects dimensions as `(Width, Height)`, whereas NumPy array shapes are ordered as `(Height, Width, Channels)`. Because I had accidentally passed `(h, w)`, OpenCV quietly dropped every frame without raising an exception. I tracked it down by checking `frame.shape` and matching it against the writer's parameters. Next time, I would write explicit assertions or unit checks to verify dimension ordering before initializing video writers.